

Software Security – Exercises

1. Is the following program correct? If no, explain the wrong point(s).

```
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char *return_val = 0;
    const size_t bufsize = strlen(argv[0]) + 1;
    char *buf = (char *)malloc(bufsize);
    if (!buf) {
        return EXIT_FAILURE;
    }
    /* ... */
    free(buf);
    /* ... */
    strcpy(buf, argv[0]);
    /* ... */
    return EXIT_SUCCESS;
}
```

2. Is the following program correct? If no, explain the wrong point(s).

```
enum { MAX_ALLOCATION = 1000 };

int main(int argc, const char *argv[]) {
    char *c_str = NULL;
    size_t len;

    if (argc == 2) {
        len = strlen(argv[1]) + 1;
        if (len > MAX_ALLOCATION) {
            /* Handle error */
        }
        c_str = (char *)malloc(len);
        if (c_str == NULL) {
            /* Handle error */
        }
        strcpy(c_str, argv[1]);
    } else {
        printf("%s\n", "usage: $>a.exe [string]");
        return EXIT_FAILURE;
    }
    free(c_str);
    return 0;
}
```

3. Is the following program correct? If no, explain the wrong point(s).

```
#include <stdlib.h>
#include <time.h>

struct tm *make_tm(int year, int mon, int day, int hour,
                  int min, int sec) {
    struct tm *tmb;
    tmb = (struct tm *)malloc(sizeof(tmb));
    if (tmb == NULL) {
        return NULL;
    }
    *tmb = (struct tm) {
        .tm_sec = sec, .tm_min = min, .tm_hour = hour,
        .tm_mday = day, .tm_mon = mon, .tm_year = year
    };
    return tmb;
}
```

4. Is the following program correct? If no, explain the wrong point(s).

```
#include <stdlib.h>

enum { BUFFER_SIZE = 32 };

int f(void) {
    static char *text_buffer = NULL;
    if (text_buffer == NULL) {
        text_buffer = (char *)malloc(BUFFER_SIZE);
        if (text_buffer == NULL) {
            return -1;
        }
    }
    return 0;
}
```

5. Is the following program correct? If no, explain the wrong point(s).

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
enum { MAX_ALLOCATION = 1000 };
int main(int argc, const char *argv[]) {
    char *c_str = NULL;
    size_t len;
    if (argc == 2) {
        len = strlen(argv[1]) + 1;
        if (len > MAX_ALLOCATION) {
            /* Handle error */
        }
        c_str = (char *)malloc(len);
        if (c_str == NULL) {
            /* Handle error */
        }
        strcpy(c_str, argv[1]);
    } else {
        c_str = "usage: $>a.exe [string]";
        printf("%s\n", c_str);
    }
    free(c_str);
    return 0;
}
```

6. Is the following program correct? If no, explain the wrong point(s).

```
#include <stddef.h>

struct flex_array_struct {
    size_t num;
    int data[];
};

void func(struct flex_array_struct *struct_a,
          struct flex_array_struct *struct_b) {
    *struct_b = *struct_a;
}
```

7. Does the compiler pad this structure to enforce alignment for 32-bit architecture? Explain why.

```
struct x_  
{  
    char a;    // 1 byte  
    int b;     // 4 bytes  
    short c;   // 2 bytes  
    char d;    // 1 byte  
} MyStruct;
```